

大模型微调实验

一、背景

测试现有资源能够微调多大的模型：

二、实验环境配置

2.1. 服务器信息

服务器一：work-5:192.168.1.1 显卡8*A800

服务器二：work-5:192.168.1.5 显卡8*A800

服务器三：work-5:192.168.1.11 显卡8*A800

服务器四：work-11:192.168.1.15 显卡8*A800（还未测试）

2.2. 在jicj用户下测试

2.3. 工程路径（每台环境和路径全部配置调整成一样）

/home/jicj/workspaces/pycharm/development/llm/project/LLaMA_Factory/LLaMA-Factory-gt-0.5.3_sft/

2.4. conda 环境

/home/jicj/soft/miniconda3/envs/llama_factory_fa/

2.5. 免密登录

```
cd ~/.ssh
```

```
ssh-keygen -t rsa
```

```
ssh-copy-id jicj@192.168.1.1
ssh-copy-id jicj@192.168.1.5
ssh-copy-id jicj@192.168.1.11
ssh-copy-id jicj@192.168.1.15
```

2.6. hosts文件配置

```
vim /etc/hosts
192.168.1.1 worker-1
192.168.1.5 worker-5
192.168.1.11 worker-11
192.168.1.15 worker-15
```

2.7. hostfile_1_5_11_15配置

```
vim hostfile_1_5_11_15
worker-1 slots=8
worker-5 slots=8
worker-11 slots=8
worker-15 slots=8
```

2.8. 配置环境变量

```
vim ~/.bashrc
```

```
export PATH=$PATH:/home/jicj/bin
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
```

2.9. 跑通流程脚本

2.9.1. 单机

```
CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 /home/jicj/soft/miniconda3/envs/llama_factory_fa/bin/deepspeed --num_gpus 8 --master_port=9901
src/train_bash.py \
  --stage sft \
  --model_name_or_path /home/jicj/models/Qwen1.5/Qwen1.5-14B-Chat/ \
  --do_train \
  --dataset select_1k \
  --template qwen \
  --finetuning_type full \
  --output_dir ./output/Qwen1.5-14B-Chat_sft \
  --per_device_train_batch_size 2 \
  --gradient_accumulation_steps 1 \
  --lr_scheduler_type cosine \
  --logging_steps 5 \
  --save_steps 1000 \
  --learning_rate 2e-6 \
```

```

--weight_decay 0. \
--adam_beta2 0.95 \
--warmup_ratio 0.03 \
--num_train_epochs 3.5 \
--plot_loss \
--bf16 \
--max_length 32768 \
--cutoff_len 32768 \
--max_new_tokens 32768 \
--flash_attn \
--deepspeed examples/full_multi_gpu/ds_z3_config.json

/home/jicj/soft/miniconda3/envs/llama_factory_fa/bin/deepspeed --include localhost:0,1,2,3,4,5,6,7 --master_port=9905 src/train_bash.py \
--stage sft \
--model_name_or_path /home/jicj/models/Qwen1.5/Qwen1.5-14B-Chat/ \
--do_train \
--dataset select_1k \
--template qwen \
--finetuning_type full \
--output_dir ./output/Qwen1.5-14B-Chat_sft \
--per_device_train_batch_size 2 \
--gradient_accumulation_steps 1 \
--lr_scheduler_type cosine \
--logging_steps 5 \
--save_steps 1000 \
--learning_rate 2e-6 \
--weight_decay 0. \
--adam_beta2 0.95 \
--warmup_ratio 0.03 \
--num_train_epochs 3.5 \
--plot_loss \
--bf16 \
--max_length 32768 \
--cutoff_len 32768 \
--max_new_tokens 32768 \
--flash_attn \
--deepspeed examples/full_multi_gpu/ds_z3_config.json

```

（两种方式主要在于指定GPU方式不同，方式一指定的GPU必须从0号卡开始，且必须顺序，方式二可以指定任意显卡）

2.9.2. 双机

```

/home/jicj/soft/miniconda3/envs/llama_factory_fa/bin/python -m torch.distributed.run \
--nproc_per_node 8 \
--nnodes 2 \
--node_rank 0 \
--master_addr worker-5 \
--master_port 9905 \
src/train_bash.py \
--deepspeed examples/full_multi_gpu/ds_z3_config.json \
--stage sft \
--do_train \
--model_name_or_path /home/jicj/models/Qwen1.5/Qwen1.5-14B-Chat/ \
--dataset meiya_to_alpaca \
--dataset_dir ./data \
--template qwen \
--finetuning_type full \
--output_dir ./output/Qwen1.5-14B-Chat_sft_ \
--overwrite_cache \
--overwrite_output_dir \
--cutoff_len 1024 \
--preprocessing_num_workers 16 \
--per_device_train_batch_size 1 \
--per_device_eval_batch_size 1 \
--gradient_accumulation_steps 2 \
--lr_scheduler_type cosine \
--logging_steps 10 \
--warmup_steps 20 \
--save_steps 100 \
--eval_steps 100 \
--evaluation_strategy steps \
--learning_rate 5e-5 \
--num_train_epochs 1.0 \
--max_samples 3000 \
--val_size 0.1 \
--ddp_timeout 1800000 \
--plot_loss \
--fp16

```

```
/home/jicj/soft/miniconda3/envs/llama_factory_fa/bin/deepspeed --hostfile ./hostfile_5_11 \
--master_port=9905 src/train_bash.py \
--stage sft \
--model_name_or_path /home/jicj/models/Qwen1.5/Qwen1.5-32B-Chat/ \
--do_train \
--dataset select_1k \
--overwrite_cache \
--overwrite_output_dir \
--template qwen \
--finetuning_type full \
--output_dir ./output/Qwen1.5-32B-Chat_sft \
--per_device_train_batch_size 1 \
--gradient_accumulation_steps 1 \
--lr_scheduler_type cosine \
--logging_steps 10 \
--save_steps 100 \
--cutoff_len 32768 \
--learning_rate 5e-6 \
--num_train_epochs 1.0 \
--warmup_steps 200 \
--plot_loss \
--bf16 \
--flash_attn \
--deepspeed examples/full_multi_gpu/ds_z3_offload_config.json
```

（方式一需要在每一台服务器上都要执行，且每台--node_rank 0 值需要按顺序改下；方式二只需要在其中一台执行即可；）

2.10. 评估显存（大家说不是很准）

```
from transformers import AutoModel
from deepspeed.runtime.zero.stage3 import estimate_zero3_model_states_mem_needs_all_live
model = AutoModel.from_pretrained("/home/jicj/models/Qwen1.5/Qwen1.5-72B-Chat")
estimate_zero3_model_states_mem_needs_all_live(model, num_gpus_per_node=8, num_nodes=2)
```

三、实验

zero主要有三个优化阶段：

- zero-1分割优化器状态：减少4倍内存，通信量和数据并行相同
- zero-2分割优化器状态+梯度：减少8倍内存，通信量与数据并行相同
- zero-3分割优化器状态+梯度+参数：内存减少与数据并行度呈线性关系。例如在64个gpu之间拆分将产生64倍的内存缩减。通信量有50%的适度增长。

zero的本质，是在数据并行的基础上，对冗余空间占用进行深度优化。

模型	Stage 2	Stage 2 + offload	Stage 3	Stage 3 + offloads	lora
Qwen-14B-Chat			1. Qwen1.5-14B-Chat14A800z3, 32k1epoch21 Qwen1.5-14B-Chat-Inode-data_1k-4xA800-ds_z3_config-cutoff_len_32k_20240513.log 2. Qwen1.5-14B-Chat单个节点数据1千条在8张A800上用z3, 长度为32k微调1个epoch时间约4分钟； Qwen1.5-14B-Chat-Inode-data_1k-8xA800-ds_z3_config-cutoff_len_32k_20240513.log 3. Qwen1.5-14B-Chat单个节点数据1千条在8张A800上用z3, 长度为32k微调1个epoch时间约4分钟； Qwen1.5-14B-Chat-Inode-data_1k-8xA800-ds_z3_config-cutoff_len_32k-no_flash_attn_20240513.log 4. 用CUDA_VISIBLE_DEVICES=0, 1, 2, 3, 4, 5, 6, 7指定显卡，实际使用的显卡都会从0号卡开始，且卡必须顺序； Qwen1.5-14B-Chat-Inode-data_1k-8xA800-ds_z3_config-cutoff_len_32k-cuda_method1_20240514.log 5. 用--include localhost:0, 1, 2, 3, 4, 5, 6, 7指定显卡，可以指定任意空闲的显卡； Qwen1.5-14B-Chat-Inode-data_1k-8xA800-ds_z3_config-cutoff_len_32k-cuda_method2_20240514.log 说明： A. 1测试Qwen1.5-14B-Chat最少需要4张A800能够进行z3全量微调； B. 1和2测试4张卡和8张卡微调速度，从21分钟提高到4分钟（测试时每张卡batch都为1，提升batch性能应该还会提升）； C. 2和3测试加flash_attn性能是否提升，由于每张卡batch都为1可能效果不明显（TODO需进一步测试）；	1. Qwen1.5-14B-Chat单个节点数据1千条在8张A800上用lora, 长度为32k微调1个epoch时间约32秒； lora_14B_batch1_data1k_len32768.log 2. Qwen1.5-32B-Chat单个节点数据1千条在8张A800上用lora, 长度为32k微调1个epoch时间约60秒； lora_32B_batch1_data1k_len32768.log 3. Qwen1.5-72B-Chat单个节点数据1千条在8张A800上用lora, 长度为32k微调1个epoch，显存溢出； lora_72B_batch1_data1k_len1024.log 4. Qwen1.5-14B-Chat两个节点数据1千条在16张A800上用lora, 长度为32k微调1个epoch时间约21秒； lora_node2_14B_batch1_data1k_len32768.log 5. Qwen1.5-32B-Chat两个节点数据1千条在16张A800上用lora, 长度为32k微调1个epoch时间约37秒； lora_node2_32B_batch1_data1k_len32768.log 6. Qwen1.5-72B-Chat两个节点数据1千条在16张A800上用lora, 长度为32k微调1个epoch，显存溢出； lora_node2_72B_batch1_data1k_len1024.log	

Qwen-32B-Chat		1. Qwen1.5-32B-Chat两个节点数据1千条在16张A800上用ds_z3_config, 长度为32k微调1个epoch时间约6分钟; Qwen1.5-32B-Chat-2node-data-1k-16xA800-ds_z3_config_20240514.log 2. Qwen1.5-32B-Chat两个节点数据1千条batch设置为8在16张A800上用ds_z3_config, 长度为32k微调1个epoch时间约1分钟多; Qwen1.5-32B-Chat-2node-data-1k-batch8-16xA800-ds_z3_config_20240514.log.log 3. Qwen1.5-32B-Chat两个节点数据1万条batch设置为4在16张A800上用ds_z3_config, 长度为32k微调1个epoch时间约19分钟多; Qwen1.5-32B-Chat-2node-data-1w-batch4-16xA800-ds_z3_config_20240515.log 4. Qwen1.5-32B-Chat两个节点数据1万条batch设置为8在16张A800上用ds_z3_config, 长度为32k微调1个epoch时间约11分钟多; Qwen1.5-32B-Chat-2node-data-1w-batch8-16xA800-ds_z3_config_20240515.log 说明: A. 相同参数ds_z3_config为6分钟, ds_z3_offload_config为35分钟, offload速度明显下降; B. 1和2微调数据为1k, batch由1变成8, 时间从约6分钟提升到约1分钟多; 3和4微调数据为1w, batch由4变成8, 时间从约19分钟提升到约11分钟多; 说明增加batch微调时间有比较明显提升;	1. Qwen1.5-32B-Chat单个节点数据1千条在1张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约18小时47分钟; Qwen1.5-32B-Chat-1node-data-1k-1xA800-ds_z3_offload_config_20240513.log 2. Qwen1.5-32B-Chat单个节点数据1千条在2张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约6小时45分钟; Qwen1.5-32B-Chat-1node-data-1k-2xA800-ds_z3_offload_config_20240513.log 3. Qwen1.5-32B-Chat单个节点数据1千条在8张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约1小时16分钟; Qwen1.5-32B-Chat-1node-data-1k-8xA800-ds_z3_offload_config_20240513.log 4. Qwen1.5-32B-Chat单个节点数据1千条在8张A800上用ds_z3_offload_optimizer_config, 长度为32k微调1个epoch时间约1小时18分钟; Qwen1.5-32B-Chat-1node-data-1k-8xA800-ds_z3_offload_optimizer_config_20240513.log 5. Qwen1.5-32B-Chat两个节点数据1千条在16张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约35分钟; Qwen1.5-32B-Chat-2node-data-1k-16xA800-ds_z3_offload_config_20240511.log	
Qwen-72B-Chat		1. Qwen1.5-72B-Chat三个节点数据1千条在24张A800上用ds_z3_config, 长度为32k微调1个epoch时间约15分钟; Qwen1.5-72B-Chat-3node-data-1k-24xA800-ds_z3_config-cutoff_len_32k_20240515.log	1. Qwen1.5-72B-Chat两个节点数据1千条在16张A800上用ds_z3_offload_optimizer_config, 长度为32k微调1个epoch时间约55分钟; Qwen1.5-72B-Chat-2node-data-1k-16xA800-ds_z3_offload_optimizer_config-cutoff_len_32k_20240514.log	
Qwen-110B-Chat		1. 三节点微调跑不起来, 用ds_z3_config方式显存不足; 2. 四节点微调, 用ds_z3_config方式, 文本长度32k长度显存不足, 文本长度1k可以微调;	1. 三节点微调跑不起来, 用ds_z3_offload_config方式cpu内存方式不足; 2. Qwen1.5-110B-Chat四个节点数据1千条在32张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约31分钟; Qwen1.5-110B-Chat-4node-data-1k-32xA800-ds_z3_offload_config-cutoff_len_32k_20240516.log	
Qwen1.5-MoE-A2.7B-Chat		1. Qwen1.5-MoE-A2.7B-Chat单个节点数据1千条在8张A800上用z3, 长度为32k微调1个epoch时间约26分钟; Qwen1.5-MoE-A2.7B-Chat-1node-data-1k-8xA800-ds_z3_config-cutoff_len_32k_20240515.log 2. Qwen1.5-MoE-A2.7B-Chat两个节点数据1千条在16张A800上用z3, 长度为32k微调1个epoch时间约14分钟多; Qwen1.5-MoE-A2.7B-Chat-2node-data-1k-16xA800-ds_z3_config-cutoff_len_32k_20240515.log		
DeepSeek-V2 (236B)		1. 四节点微调跑不起来, 用ds_z3_config方式显存不足;	1. 四节点微调跑不起来, 用ds_z3_offload_config方式cpu内存方式不足;	

实验结论:

3.1. 单卡最大可以微调多大模型;

Stage 3 + offloads: 1. Qwen1.5-32B-Chat单个节点数据1千条在1张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约18小时47分钟;

3.2. 单台服务器最大能跑多大的模型;

Stage 3: 1. Qwen1.5-14B-Chat单个节点数据1千条在4张A800上用z3, 长度为32k微调1个epoch时间约21分钟;

Stage 3 + offloads: 5. Qwen1.5-32B-Chat两个节点数据1千条在16张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约35分钟;

3.3. 两台服务器最大能跑多大的模型;

Stage 3: 1. Qwen1.5-32B-Chat两个节点数据1千条在16张A800上用ds_z3_config, 长度为32k微调1个epoch时间约6分钟;

Stage 3 + offloads: 1. Qwen1.5-72B-Chat两个节点数据1千条在16张A800上用ds_z3_offload_optimizer_config, 长度为32k微调1个epoch时间约55分钟;

3.4. 两台相比单台跑同样的模型性能提升多少;

Stage 3 + offloads: 3. Qwen1.5-32B-Chat单个节点数据1千条在8张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约1小时16分钟;

Stage 3 + offloads: 5. Qwen1.5-32B-Chat两个节点数据1千条在16张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约35分钟;

3.5. 三台最大能跑多大模型;

Stage 3: 1. Qwen1.5-72B-Chat三个节点数据1千条在24张A800上用ds_z3_config, 长度为32k微调1个epoch时间约15分钟;

3.6. 四台最大能跑多大模型;

Stage 3 + offloads: 2. Qwen1.5-110B-Chat四个节点数据1千条在32张A800上用ds_z3_offload_config, 长度为32k微调1个epoch时间约31分钟;

3.7. 微调时间比对

不同大小模型（方式：z3；节点：4个；Batch：8；数据：1k；文本长度：32k；）

模型	Qwen1.5-14B-Chat	Qwen1.5-32B-Chat	Qwen1.5-72B-Chat	Qwen1.5-110B-Chat
4个节点	31秒	52秒	1分51秒	Batch 8：文本长度32k跑步起来；Batch 1：文本长度1k可以跑；

不同节点（方式：z3；模型大小：14B；Batch：8；数据：1w；文本长度：32k；）

节点	1节点8卡	2节点16卡	3节点24卡	4节点32卡
Qwen1.5-14B-Chat	8分27秒	5分21秒	5分29秒	3分21秒

不同batch（方式：z3；节点：1个；模型大小：14B；数据：1k；文本长度：32k；）

Batch	Batch1	Batch2	Batch4	Batch8	Batch16
Qwen1.5-14B-Chat	2分29秒	1分22秒	54秒	51秒	1分2秒

不同数据（方式：z3；节点：1个；模型大小：14B；Batch：8；文本长度：32k；）

数据	1k	2k	1w	2w
Qwen1.5-14B-Chat	56秒	1分42秒	8分27秒	异常

不同方式（节点：1个；模型大小：14B；Batch：8；数据：2k；文本长度：32k；）

method	z2	z2+offload	z3	z3+offload
Qwen1.5-14B-Chat	1分36秒	2分17秒	56秒	5分40秒

不同文本长度（方式：z3；节点：1个；模型大小：14B；Batch：8；数据：1w；）#TODO 不同长度微调时间并没有太大变化，需要再排查下原因

文本长度	1k	2k	4k	8k	16k	32k
Qwen1.5-14B-Chat	8分18秒	8分28秒	8分30秒	8分29秒	8分29秒	8分27秒

四、其它

如何选择最佳性能的ZeRO阶段和offload方式

一般而言，以下规则适用（与实测可能存在偏差，具体用时可以根据实际场景再测试对比下）：

从速度角度来看 **Stage 0 (DDP) > Stage 1 > Stage 2 > Stage 2 + offload > Stage 3 > Stage 3 + offloads**

从GPU内存使用角度来看 **Stage 0 (DDP) < Stage 1 < Stage 2 < Stage 2 + offload < Stage 3 < Stage 3 + offloads**

因此，当想要在适合最少数量的GPU的情况下获得最快的执行速度时，可以遵循以下过程。我们从最快的方法开始，如果遇到GPU内存不足，然后转到下一个速度较慢但使用更少GPU内存的方法，依此类推。

- 1. 将batch_size设置为1，通过梯度累积实现任意的有效batch_size
- 2. 如果OOM则，设置--gradient_checkpointing 1（HF Trainer），或者 model.gradient_checkpointing_enable()
- 3. 如果OOM则，尝试ZeRO stage 2
- 4. 如果OOM则，尝试ZeRO stage 2 + offload_optimizer
- 5. 如果OOM则，尝试ZeRO stage 3
- 6. 如果OOM则，尝试offload_param到CPU
- 7. 如果OOM则，尝试offload_optimizer到CPU
- 8. 如果OOM则，尝试降低一些默认参数。比如使用generate时，减小beam_search的搜索范围
- 9. 如果OOM则，使用混合精度训练，在Ampere的GPU上使用bf16，在旧版本GPU上使用fp16
- 10. 如果仍然OOM，则使用ZeRO-Infinity，使用offload_param和offload_optimizer到NVME

性能优化

- 1. 一旦使用batch_size=1时，没有导致OOM，测量此时的有效吞吐量，然后尽可能增大batch_size，因为批量大小越大，GPU的效率越高，因为它们在做乘法的矩阵很大时表现最佳。

2. 开始优化参数，可以关闭offload参数，或者降低ZeRO stage，然后调整batch_size，然后继续测量吞吐量，直到性能比较满意（调参可以增加66%的性能）

一些其他建议

1. 如果训模型from scratch, hidden size最好可以被16整除
2. batch size最好可以被2整除

五、实验过程遇到的问题记录

5.1. 如果报pdsh没安装

安装步骤如下：

```
tar -jxvf pdsh-2.29.tar.bz2
cd pdsh-2.29/
./configure --with-ssh --enable-static-modules --prefix=/home/jicj && make && make install
vim ~/.bashrc
export PATH=$PATH:/home/jicj/bin
source ~/.bashrc
sudo ln -s /home/jicj/bin/pdsh /usr/bin/
pdsh -V
```

5.2. 采用offload

#报错

```
worker-5: File "/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/utils/cpp_extension.py", line
1649, in verify_ninja_availability
worker-5:     raise RuntimeError("Ninja is required to load C++ extensions")
worker-5: RuntimeError: Ninja is required to load C++ extensions
worker-5: File "/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/deepspeed-0.13.1-py3.10.egg/deepspeed
/ops/adam/cpu_adam.py", line 102, in __del__
worker-5:     self.ds_opt_adam.destroy_adam(self.opt_id)
worker-5: AttributeError: 'DeepSpeedCPUAdam' object has no attribute 'ds_opt_adam'
```

#

```
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH
source ~/.bashrc
```

#以上修改错误依旧

修正ninja中的python路径（错误依旧）

```
vim /home/jicj/soft/miniconda3/envs/llama_factory_fa/bin/ninja
```

#配置ninja环境变量（错误依旧）

```
vim ~/.bashrc
export PATH=/home/jicj/soft/miniconda3/envs/llama_factory_fa/bin:$PATH
source ~/.bashrc
```

#

安装ninja（错误依旧）

```
wget https://github.com/ninja-build/ninja/releases/download/v1.11.1/ninja-linux.zip
sudo unzip ninja-linux.zip -d /usr/local/bin/
sudo update-alternatives --install /usr/bin/ninja ninja /usr/local/bin/ninja 1 --force
```

#还原以上（配置ninja环境变量）（安装ninja）这两步

在.5上 /usr/local/bin/pip install ninja

scp -r worker-11:/usr/local/bin/ninja /usr/local/bin/（.11上原来就有ninja,这个ninja和安装ninja的ninja文件内容不一样，.11上的ninja文件内容如下）

```
#!/usr/bin/python3
```

```
# -*- coding: utf-8 -*-
```

```
import re
```

```
import sys
```

```
from ninja import ninja
```

```
if __name__ == '__main__':
```

```
    sys.argv[0] = re.sub(r'(-script|.pyw|\.exe)?$', '', sys.argv[0])
```

```
    sys.exit(ninja())
```

#

#还原ninja路径

```
export PATH=/usr/local/bin:$PATH
```

```
which ninja
```

5.3. 包如下错

```

Allowing ninja to set a default number of workers... (overridable by setting the environment variable MAX_JOBS=N)
[1/4] /usr/local/cuda/bin/bin/nvcc -DTORCH_EXTENSION_NAME=cpu_adam -DTORCH_API_INCLUDE_EXTENSION_H -DPYBIND11_COMPILER_TYPE=\\"gcc\\" -DPYBIND11_STDLIB=\\"libstdc++\\" -DPYBIND11_BUILD_ABI=\\"cxxabi1011\\" -I/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/deepspeed-0.13.1-py3.10.egg/deepspeed/ops/csrc/includes -I/usr/local/cuda/bin/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/torch/csrc/api/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/TH -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/THC -isystem /usr/local/cuda/bin/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/include/python3.10 -D_GLIBCXX_USE_CXX11_ABI=0 -D_CUDA_NO_HALF_OPERATORS -D_CUDA_NO_HALF_CONVERSIONS -D_CUDA_NO_BFLOAT16_CONVERSIONS -D_CUDA_NO_HALF2_OPERATORS --expt-relaxed-constexpr -gencode=arch=compute_80,code=compute_80 -gencode=arch=compute_80,code=sm_80 --compiler-options '-fPIC' -O3 --use_fast_math -std=c++17 -U_CUDA_NO_HALF_OPERATORS -U_CUDA_NO_HALF_CONVERSIONS -U_CUDA_NO_HALF2_OPERATORS -gencode=arch=compute_80,code=sm_80 -gencode=arch=compute_80,code=compute_80 -DBF16_AVAILABLE -U_CUDA_NO_BFLOAT16_OPERATORS -U_CUDA_NO_BFLOAT162_OPERATORS -c /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/deepspeed-0.13.1-py3.10.egg/deepspeed/ops/csrc/common/custom_cuda_kernel.cu -o custom_cuda_kernel.cu.o
FAILED: custom_cuda_kernel.cu.o
/usr/local/cuda/bin/bin/nvcc -DTORCH_EXTENSION_NAME=cpu_adam -DTORCH_API_INCLUDE_EXTENSION_H -DPYBIND11_COMPILER_TYPE=\\"gcc\\" -DPYBIND11_STDLIB=\\"libstdc++\\" -DPYBIND11_BUILD_ABI=\\"cxxabi1011\\" -I/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/deepspeed-0.13.1-py3.10.egg/deepspeed/ops/csrc/includes -I/usr/local/cuda/bin/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/torch/csrc/api/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/TH -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/THC -isystem /usr/local/cuda/bin/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/include/python3.10 -D_GLIBCXX_USE_CXX11_ABI=0 -D_CUDA_NO_HALF_OPERATORS -D_CUDA_NO_HALF_CONVERSIONS -D_CUDA_NO_BFLOAT16_CONVERSIONS -D_CUDA_NO_HALF2_OPERATORS --expt-relaxed-constexpr -gencode=arch=compute_80,code=compute_80 -gencode=arch=compute_80,code=sm_80 --compiler-options '-fPIC' -O3 --use_fast_math -std=c++17 -U_CUDA_NO_HALF_OPERATORS -U_CUDA_NO_HALF_CONVERSIONS -U_CUDA_NO_HALF2_OPERATORS -gencode=arch=compute_80,code=sm_80 -gencode=arch=compute_80,code=compute_80 -DBF16_AVAILABLE -U_CUDA_NO_BFLOAT16_OPERATORS -U_CUDA_NO_BFLOAT162_OPERATORS -c /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/deepspeed-0.13.1-py3.10.egg/deepspeed/ops/csrc/common/custom_cuda_kernel.cu -o custom_cuda_kernel.cu.o
<command-line>: fatal error: cuda_runtime.h: No such file or directory

```

#梳理环境变量

```

export PATH=/bin:/home/jicj/bin:/home/jicj/soft/miniconda3/bin:/home/jicj/soft/miniconda3/condabin:/home/jicj/soft/miniconda3/envs/llama_factory_fa/bin:/sbin:/snap/bin:/usr/bin:/usr/games:/usr/local/bin:/usr/local/cuda/bin:/usr/local/games:/usr/local/sbin:/usr/sbin:/usr/local/cuda/targets/x86_64-linux/include#

```

#/usr/local/cuda/bin/bin/nvcc中多了一个bin,是11这台服务器多了一层,将其删掉

```

[1/4] /usr/local/cuda/bin/bin/nvcc -DTORCH_EXTENSION_NAME=cpu_adam -DTORCH_API_INCLUDE_EXTENSION_H -DPYBIND11_COMPILER_TYPE=\\"gcc\\" -DPYBIND11_STDLIB=\\"libstdc++\\" -DPYBIND11_BUILD_ABI=\\"cxxabi1011\\" -I/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/deepspeed-0.13.1-py3.10.egg/deepspeed/ops/csrc/includes -I/usr/local/cuda/bin/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/torch/csrc/api/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/TH -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/torch/include/THC -isystem /usr/local/cuda/bin/include -isystem /home/jicj/soft/miniconda3/envs/llama_factory_fa/include/python3.10 -D_GLIBCXX_USE_CXX11_ABI=0 -D_CUDA_NO_HALF_OPERATORS -D_CUDA_NO_HALF_CONVERSIONS -D_CUDA_NO_BFLOAT16_CONVERSIONS -D_CUDA_NO_HALF2_OPERATORS --expt-relaxed-constexpr -gencode=arch=compute_80,code=compute_80 -gencode=arch=compute_80,code=sm_80 --compiler-options '-fPIC' -O3 --use_fast_math -std=c++17 -U_CUDA_NO_HALF_OPERATORS -U_CUDA_NO_HALF_CONVERSIONS -U_CUDA_NO_HALF2_OPERATORS -gencode=arch=compute_80,code=sm_80 -gencode=arch=compute_80,code=compute_80 -DBF16_AVAILABLE -U_CUDA_NO_BFLOAT16_OPERATORS -U_CUDA_NO_BFLOAT162_OPERATORS -c /home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/deepspeed-0.13.1-py3.10.egg/deepspeed/ops/csrc/common/custom_cuda_kernel.cu -o custom_cuda_kernel.cu.o

```

#报错

```

File "/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/site-packages/deepspeed-0.13.1-py3.10.egg/deepspeed/ops/

```

```

/op_builder/builder.py", line 52, in installed_cuda_version

```

```

    output = subprocess.check_output([cuda_home + "/bin/nvcc", "-V"], universal_newlines=True)

```

```

File "/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/subprocess.py", line 420, in check_output

```

```

    return run(*popenargs, stdout=PIPE, timeout=timeout, check=True,

```

```

File "/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/subprocess.py", line 501, in run

```

```

    with Popen(*popenargs, **kwargs) as process:

```

```

File "/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/subprocess.py", line 966, in __init__

```

```

    self._execute_child(args, executable, preexec_fn, close_fds,

```

```

File "/home/jicj/soft/miniconda3/envs/llama_factory_fa/lib/python3.10/subprocess.py", line 1842, in _execute_child

```

```

    raise child_exception_type(errno_num, err_msg, err_filename)

```

```

FileNotFoundError: [Errno 2] No such file or directory: '/usr/local/cuda/bin/bin/nvcc'

```

#

添加CUDA_HOME, 上面PATH最后面的/usr/local/cuda/targets/x86_64-linux/include应该可以不用

```

export CUDA_HOME=/usr/local/cuda

```

```

export PATH=$CUDA_HOME:$PATH

```

5.4.#.1网卡问题

```

cd /etc/netplan/

```

```

sudo vim 00-installer-config.yaml

```

将报错的网卡改成:

```

ens17f0:

```

```

    dhcp4: true

```

5.5.#.11路径不一致问题

建立软连接

```

sudo ln -s /mnt/nvme0n1p1 /mnt/nvme0n1

```

```

sudo ln -s /mnt/nvme1n1p1 /mnt/nvme1n1

```

```

sudo ln -s /mnt/nvme2n1p1 /mnt/nvme2n1

```

```

sudo ln -s /mnt/nvme3n1p1 /mnt/nvme3n1

```